

# MACHINE LEARNING IN FINANCE

## MODULE 6 | LESSON 2

---

# APPLICATION OF DEEP LEARNING IN PREDICTING YIELD CURVES

---

|                        |                                                           |
|------------------------|-----------------------------------------------------------|
| <b>Reading Time</b>    | 100 minutes                                               |
| <b>Prior Knowledge</b> | Machine learning, Introduction to deep learning           |
| <b>Keywords</b>        | Yield curve, Logistic Regression, Multi-Linear Perceptron |

---

*In the previous lesson, we introduced the concept of deep learning and its applications. In this lesson, we will build on deep learning concepts by solving a real-world problem of forecasting yield curves using deep learning algorithms and comparing our results to a high-order polynomial regression.*

## 1. Recap of Yield Curves

We can define yield curves as a graph showing interest rates (bonds) having different maturity rates. Yield curves are important in the financial world for these reasons:

1. They are used as a benchmark for setting mortgage rates, and;
2. They provide a good indicator for predicting economic growth.

There are three types of yield curves:

- Normal yield curve
- Inverted yield curve
- Flat yield curve

We will review each one of them below.

### 1.1 Normal Yield Curve

The graph of a normal yield curve shows an upward trajectory that implies longer maturity date bonds will pay more in interest (have higher yield) as compared to shorter maturity date bonds.

A normal yield curve can be interpreted as people having confidence that the economy will perform well in the future, and we would therefore expect the central banks to increase the interest rate in that future date. The central banks raise interest rates in response to the inflationary pressures caused by an expanding economy.

## 1.2 Inverted Yield Curve

An inverted yield curve is a downward sloping curve that implies a recession is about to happen. An inverted yield curve means that people believe interest rates will be lower than they currently are, and we therefore expect the central banks to cut interest rates so as to encourage borrowing of more money by investors, consumers, and businesses. This in turn leads to economic growth.

## 1.3 Flat Yield Curve

We get a flat yield curve when an economy is shifting from one state to another. For example, when an expanding economy starts contracting we see that a yield curve that was initially normal begins to flatten due to people expecting an economic downturn and thus a drop in long-term interest rates. The opposite is also true.

Now that we have a grasp of how useful yield curves are, in the subsequent sections, we are going to forecast yield curves using conventional machine learning algorithms and a deep learning algorithm.

# 2. Forecasting Yield Curves

This case study is a time series problem whose foundation we studied in the Econometrics course. It is also a supervised machine learning problem since we have outputs that our model will be learning from. The predicted variables in this case will be multiple outputs.

To solve this problem, we will follow an industry standard methodology: Cross-Industry Process for Data Mining (CRISP-DM) used for solving data science problems.

With the exception of the deployment stage, which is left for curious students to practice on their own, the other steps will be demonstrated here.

## 2.1 Problem Understanding

The focus of this study is to find and understand factors that affect movements of the yield curve. Among the variables we will use are the historical data of the yield curve, federal debt percentage that is held by investors, and the corporate spread. The dependent variable will be the 1-month, 5-year and 30-year tenure of the yield curve.

We sourced our data from the Yahoo and FRED websites. In this problem, we will use polynomials of various degrees to forecast the yield curves.

## 2.2 Data Understanding

We start by loading packages that we will use throughout this lesson as shown below.

```
In [ ]: # Load Libraries
# Global Libraries
# Disable the warnings
import warnings

# Time series Models
from datetime import datetime

import numpy as np
import pandas as pd
import pandas_datareader.data as web

# Plotting
import seaborn as sns

# Libraries for Statistical Models
import statsmodels.api as sm
from matplotlib import pyplot
from pandas.plotting import scatter_matrix

# Regression ML
from sklearn.linear_model import LinearRegression

# Loss Function
from sklearn.metrics import mean_squared_error

# Modeling
from sklearn.model_selection import (
    GridSearchCV,
    KFold,
    cross_val_score,
    train_test_split,
)
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import PolynomialFeatures

warnings.filterwarnings("ignore")
```

Next, we load the data using the pandas `datareader` function:

```
In [ ]: # downloading the data
start = datetime(2007, 6, 30)
end = datetime(2022, 6, 30)

data = [
    "DGS1M0",
    "DGS3M0",
    "DGS1",
    "DGS2",
    "DGS5",
    "DGS7",
    "DGS10",
    "DGS30",
    "TREAST", # -- U.S. Treasury securities held by the Federal Reserve ( Milli
    "FYGFDPU", # -- Federal Debt Held by the Public ( Millions of Dollars )
    "FDHBFIN", # -- Federal Debt Held by Foreign and International Investors (
    "GFDEBTN", # -- Federal Debt: Total Public Debt ( Millions of Dollars )
    "BAA10Y", # -- Baa Corporate Bond Yield Relative to Yield on 10-Year
]
```

```

data = web.DataReader(data, "fred", start, end).dropna(how="all").ffill()
data["FDHBFIN"] = data["FDHBFIN"] * 1000
data["GOV_PCT"] = data["TREAST"] / data["GFDEBTN"]
data["HOM_PCT"] = data["FYGFDPUN"] / data["GFDEBTN"]
data["FOR_PCT"] = data["FDHBFIN"] / data["GFDEBTN"]

```

We then define independent and dependent variables as mentioned in the previous subsection, and we also make an assumption that trading only happens on weekdays. As such, we will calculate the lagged version of the variables using a 5 trading day lag.

```

In [ ]: data.rename(
    columns={
        "DGS1M0": "1m",
        "DGS3M0": "3m",
        "DGS1": "1y",
        "DGS2": "2y",
        "DGS5": "5y",
        "DGS7": "7y",
        "DGS10": "10y",
        "DGS30": "30y",
    },
    inplace=True,
)
return_period = 5

Y = data.loc[:, ["1m", "5y", "30y"]].shift(-return_period)
Y.columns = [col + "_pred" for col in Y.columns]

X = data.loc[
    :,
    [
        "1m",
        "3m",
        "1y",
        "2y",
        "5y",
        "7y",
        "10y",
        "30y",
        "GOV_PCT",
        "HOM_PCT",
        "FOR_PCT",
        "BAA10Y",
    ],
]

df = pd.concat([Y, X], axis=1).dropna().iloc[:, :return_period, :]
Y = df.loc[:, Y.columns]
X = df.loc[:, X.columns]

df.head(10)

```

## 2.3 Exploratory Data Analysis (EDA)

EDA allows us to understand the hidden structure of our dataset by visualizing trends, relationships, and patterns that would have escaped the human eye. With EDA, we can already start to draw conclusions from our data. Exploring data constitutes a crucial step

in our data analysis. In this subsection, we will show graphical exploratory data analysis that involves taking data from the dataframe and representing it graphically. Students are required to study the graphs closely to understand what they mean.

```
In [ ]: df.shape
```

This data has 243 observations and 15 variables.

```
In [ ]: df.describe()
```

We now plot the predicted variables to observe their behavior.

```
In [ ]: # print('\033[1m' + 'Fig 5: Yield Curve Prediction' + '\033[0m' )
Y.plot(style=["-", "--", "*"])
pyplot.suptitle(
    "Fig. 5: Yield Curve Prediction",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

Notice that in the beginning of 2017, deviations for the 1-month, 5-year, and 30-year rates were high but steadily reduced afterwards. As of 2022, the medium- and long-term rates, that is 5 years and 30 years, were approximately equal but very much deviated from the short-term rate. Therefore, these time series trends show non-stationary behavior.

```
In [ ]: # Scatterplot Matrix
pyplot.figure(figsize=(15, 15))
scatter_matrix(df, figsize=(15, 16))
pyplot.suptitle(
    "Fig. 6: Scatter Plot Matrix",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

From the above scatter plots, it is evident that there exists a strong linear relationship between the predicted variables and other tenures in the dataset.

```
In [ ]: # correlation
correlation = df.corr()
pyplot.figure(figsize=(16, 16))
pyplot.title("Correlation Matrix")
pyplot.suptitle(
    "Fig. 7: Correlation Plot",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
sns.heatmap(correlation, vmax=1, square=True, annot=True, cmap="cubehelix")
pyplot.show()
```

The correlation plot above can be interpreted in a similar manner as the scatter plot figure discussed earlier.

We now do an EDA on the predicted variables as a time series plot.

First is the 1-month time series analysis.

```
In [ ]: temp_Y = df["1m_pred"]
res = sm.tsa.seasonal_decompose(temp_Y, period=52)
fig = res.plot()
fig.set_figheight(8)
fig.set_figwidth(15)
pyplot.suptitle(
    "Fig. 8: Time Series Trend Analysis for 1 month period",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

Then, the 5-year analysis:

```
In [ ]: temp_Y = df["5y_pred"]
res = sm.tsa.seasonal_decompose(temp_Y, period=52)
fig = res.plot()
fig.set_figheight(8)
fig.set_figwidth(15)
pyplot.suptitle(
    "Fig. 9: Time Series Trend Analysis of the 5-Year predicted variable",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

Finally, the 30-year variable

```
In [ ]: temp_Y = df["30y_pred"]
res = sm.tsa.seasonal_decompose(temp_Y, period=52)
fig = res.plot()
fig.set_figheight(8)
fig.set_figwidth(15)
pyplot.suptitle(
    "Fig. 10: Time Series Trend Analysis of the 30-Year predicted variable",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()
```

### 3. Data Preparation

The dataset in this exercise does not need much preprocessing, and therefore, we are going to use it as is. Note that data preparation involves transforming the data, for

example, by scaling so that we remove bias when running machine learning algorithms on our datasets.

## 4. Modeling

### 4.1 Splitting the Dataset into Train and Test Samples

We will split the data into 80% for training (modeling) the dataset and 20% to validate our model. To evaluate our algorithms, we choose the mean squared error since this is a regression exercise.

```
In [ ]: # split out validation dataset for model validation
test_size = 0.2

seed = 42
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=test_size, random_state=seed
)
```

We use cross-validation to validate the model while training on the limited data sample.

```
In [ ]: # test options for regression
num_folds = 10
scoring = "neg_mean_squared_error"
```

### 4.2 Forecasting Yield Curves: Fitting High Order Polynomials

Linear regression assumes that there exists a linear relationship between the dependent and independent variables. To account for non-linearity in our dataset, we choose to employ polynomial regression by adding polynomial terms:  $y = w_1 x + w_2 x^2 + \dots + w_m x^m + b$  where  $m$  is the degree of our polynomial.

In our case study, we will compare the performance of 1-order, 2-order, and 3-order polynomial regressions. We use transformer `PolynomialFeatures` from scikit-learn to add quadratic terms.

```
In [ ]: lr = LinearRegression()
lr.fit(X_train, y_train)

pred = lr.predict(X_test)

MSE = mean_squared_error(y_test, pred)

print("linear regression mean squared error:", MSE)
```

```
In [ ]: # Quadratic
poly = PolynomialFeatures(degree=2)
X_train2 = poly.fit_transform(X_train)
X_test2 = poly.fit_transform(X_test)

model = lr.fit(X_train2, y_train)
```

```

print(
    "quadratic regression mean squared error:",
    mean_squared_error(y_test, model.predict(X_test2)),
)

# Cubic
poly = PolynomialFeatures(degree=3)
X_train3 = poly.fit_transform(X_train)
X_test3 = poly.fit_transform(X_test)

model = lr.fit(X_train3, y_train)
print(
    "degree 3 polynomial regression mean squared error:",
    mean_squared_error(y_test, model.predict(X_test3)),
)

```

Clearly, linear regression outperforms higher-order polynomials in the prediction task, and we would therefore use cross-validation with it to compare our output with a deep learning algorithm.

### 4.3 Forecasting Yield Curves: Cross-Validation

In  $k$ -fold cross-validation, data is split randomly into  $k$  folds and without replacement. We train our data into  $k-1$  folds and validate the model on the remaining one fold also called a test fold. The method is repeated  $k$ -terms, meaning we will have  $k$  results.  $k$ -fold cross-validation is typically used for tuning our model.

We estimate the average performance  $E$  by  $E = \frac{1}{k} \sum_{i=1}^k E_i$

Cross-validation can be applied to any model, and we will therefore first discuss neural networks in the next subsection and apply cross-validation in our neural network algorithm to fine-tune its performance.

### 4.4 Forecasting Yield Curves - Backpropagation

In the last lesson, we studied backpropagation in detail and we will put our skills to practice in this section. We will later apply an artificial neural network algorithm, (MLP), which is part of the machine learning library in `sklearn`.

Before we proceed, let us familiarize ourselves with some common terminology in deep learning:

- **Forward pass:** It is synonymous with forward propagation, that is, moving from the input layer to the output layer.
- **Backward pass:** It is synonymous with backward propagation, that is, moving from the output layer to the input layer.
- **Epoch:** It is the number of times our model trains the whole data, that is, one epoch is equivalent to one forward pass and one backward pass for the training data.
- **Iterations:** Number of times we perform forward pass and backward pass.
- **Batch size:** It is the subset of training data that we use in one forward pass and one backward pass.

In our model, we will use a two-layered neural network where the input layer has twelve variables, that is, the predictor variables. The hidden layer has 20 nodes for purposes of demonstration, and the output layer has three nodes corresponding to the predicted variables. So we start by defining the number of nodes in each layer.

```
In [ ]: from nnv import NNV

print("\033[1m" + "Fig 12: A Two Layer Neural Network" + "\033[0m")
layersList = [
    {"title": "inputs \n + bias", "units": 12, "color": "darkBlue"},
    {"title": "hidden layer", "units": 20},
    # {"title": "$f(z)$", "units": 1, "edges_color": "red", "edges_width": 2},
    {"title": "output", "units": 3, "color": "darkBlue"},
]

NNV(layersList).render(save_to_file="my_example.png")
pyplot.show()
```

```
In [ ]: num_input = 12
num_hidden = 20
num_output = 3
```

Next, we initialize values of weights and bias randomly. We start by initializing the input to hidden layer weights.

```
In [ ]: W_x_h = np.random.randn(num_input, num_hidden)
b_h = np.zeros((1, num_hidden))
```

Then, we initialize the hidden layer to output layer weights and bias.

```
In [ ]: W_h_y = np.random.randn(num_hidden, num_output)
b_y = np.zeros((1, num_output))
```

We can now define our activation function and its derivative, in this case, the sigmoid function.

```
In [ ]: def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    return np.exp(-z) / ((1 + np.exp(-z)) ** 2)
```

As in the previous section, we then write forward propagation and backward propagation functions.

```
In [ ]: def forward_prop(X, W_x_h, W_h_y):
    z_1 = np.dot(X, W_x_h) + b_h
    a_1 = sigmoid(z_1)
    z_2 = np.dot(a_1, W_h_y) + b_y
    y_hat = sigmoid(z_2)

    return z_1, a_1, z_2, y_hat
```

```
def back_prop(y_hat, z_1, a_1, z_2):
    delta2 = np.multiply(-(y_train - y_hat), sigmoid_derivative(z_2))
    dJ_d_W_h_y = np.dot(a_1.T, delta2)
    delta1 = np.dot(delta2, W_h_y.T) * sigmoid_derivative(z_1)
    dJ_d_W_x_h = np.dot(X_train.T, delta1)

    return dJ_d_W_x_h, dJ_d_W_h_y
```

The cost function we use is the squared error as shown below:

```
In [ ]: def cost_function(y, y_pred):
        return 0.5 * np.sum((y - y_pred) ** 2)
```

Before training our model, we set the learning rate and number of iterations.

```
In [ ]: alpha = 0.01
        num_iterations = 10000
```

We now train the neural network.

```
In [ ]: cost = []

        for i in range(num_iterations):
            z_1, a_1, z_2, y_hat = forward_prop(X_train, W_x_h, W_h_y)
            dJ_d_W_x_h, dJ_d_W_h_y = back_prop(y_hat, z_1, a_1, z_2)

            # update weights
            W_x_h = W_x_h - alpha * dJ_d_W_x_h
            W_h_y = W_h_y - alpha * dJ_d_W_h_y

            # compute cost
            c = cost_function(y_train, y_hat)

            cost.append(c)
```

The plot of the cost function is presented below:

```
In [ ]: pyplot.grid()
        pyplot.plot(range(num_iterations), cost)

        pyplot.title("Cost Function")
        pyplot.xlabel("Training Iterations")
        pyplot.ylabel("Cost")
        pyplot.suptitle(
            "Fig. 13: Cost Function Plot",
            fontweight="bold",
            horizontalalignment="right",
            fontsize=15,
        )
        pyplot.show()
```

As compared to the regression models in the first subsection of this section, the neural network model performs poorly. We can always improve this performance by increasing the number of nodes in the hidden layer or increasing the number of hidden layers to allow for complex modeling of the data.

Now that we have a clear understanding of how neural networks work, let's implement a neural network algorithm called **Multilayer Perceptron** to predict yield curves.

We will use Multilayer Perceptron (MLP) as an example of a deep neural network that is fully connected.

As discussed above, an MLP algorithm follows these steps:

1. Given an input layer, training data is forward-propagated through a deep neural network to get an output.
2. Then, calculate the loss to check whether it is at a minimum.
3. The loss is backpropagated and the model updated.

Finally, once we are satisfied with the output of the above three steps, we use forward propagation to get our final output.

Below is a demonstration of the use of cross-validation and backpropagation in forecasting yield curves.

```
In [ ]: # test options for regression
num_folds = 10
scoring = "neg_mean_squared_error"
```

```
In [ ]: # spot check the algorithms
models = []

# Linear Regression Algorithm
models.append(("LR", LinearRegression()))

# Deep Learning Algorithm
models.append(("MLP", MLPRegressor()))
```

```
In [ ]: kfold_results = []
names = []
validation_results = []
train_results = []
for name, model in models:
    kfold = KFold(n_splits=num_folds)
    # converted mean square error to positive. The lower the better
    cv_results = -1 * cross_val_score(
        model, X_train, y_train, cv=kfold, scoring=scoring
    )
    kfold_results.append(cv_results)
    names.append(name)

    # Finally we Train on the full period and test against validation
    res = model.fit(X_train, y_train)
    validation_result = np.mean(np.square(res.predict(X_test) - y_test))
    validation_results.append(validation_result)
    train_result = np.mean(np.square(res.predict(X_train) - y_train))
    train_results.append(train_result)

    msg = (
        "%s: \nAverage CV error: %s \nStd CV Error: (%s) \nTraining Error:\n%s \n"
        % (
```

```

        name,
        str(cv_results.mean()),
        str(cv_results.std()),
        str(train_result),
        str(validation_result),
    )
)
print(msg)
print("-----")

```

Now let's compare performance of the two algorithms.

```

In [ ]: # compare algorithms
names = ["Linear Regression", "Multilinear Perceptron"]
fig = pyplot.figure()
# print('\033[1m' + 'Fig 14: Box Plot Comparing Model Performance' + '\033[0m' )
fig.suptitle("Algorithm Comparison")
ax = fig.add_subplot(111)
pyplot.boxplot(kfold_results)
ax.set_xticklabels(names)
fig.set_size_inches(15, 8)
pyplot.ylabel("Model Errors")
pyplot.xlabel("Models")
pyplot.suptitle(
    "Fig. 14: Box Plot Comparing Model Performance",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)
pyplot.show()

```

```

In [ ]: # compare algorithms
fig = pyplot.figure()

ind = np.arange(len(names)) # the x locations for the groups
width = 0.35 # the width of the bars

fig.suptitle("Algorithm Comparison")
ax = fig.add_subplot(111)
pyplot.bar(
    ind - width / 2, [x.mean() for x in train_results], width=width, label="Train Error",
)
pyplot.bar(
    ind + width / 2,
    [x.mean() for x in validation_results],
    width=width,
    label="Validation Error",
)
fig.set_size_inches(15, 8)
pyplot.legend()
pyplot.ylabel("Error")
pyplot.xlabel("Models")
ax.set_xticks(ind)
ax.set_xticklabels(names)
pyplot.suptitle(
    "Fig. 15: Bar Plot Comparing Training and Validation Error",
    fontweight="bold",
    horizontalalignment="right",
    fontsize=15,
)

```

```
)
pyplot.show()
```

The linear regression model has fewer errors compared to MLP despite its simplicity. To improve the performance of the MLP regressor, we seek the best parameters by use of grid search as shown below.

```
In [ ]: # Grid search : `MLPRegressor`
        """
        hidden_layer_sizes : tuple, length = n_layers - 2, default (100,)
            The ith element represents the number of neurons in the ith
            hidden layer.
        """
        param_grid = {"hidden_layer_sizes": [(20,), (50,), (20, 20), (20, 30, 20)]}
        model = MLPRegressor()
        kfold = KFold(n_splits=num_folds)
        grid = GridSearchCV(estimator=model, param_grid=param_grid, scoring=scoring, cv=
        grid_result = grid.fit(X_train, y_train)
        print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
        means = grid_result.cv_results_["mean_test_score"]
```

```
In [ ]: # prepare model
        model = MLPRegressor(hidden_layer_sizes=(20, 30, 20))
        model.fit(X_train, y_train)
```

```
In [ ]: # estimate accuracy on validation set
        from sklearn.metrics import mean_squared_error, r2_score

        predictions = model.predict(X_test)
        mse_MLP = mean_squared_error(y_test, predictions)
        r2_MLP = r2_score(y_test, predictions)

        # prepare model
        model_2 = LinearRegression()
        model_2.fit(X_train, y_train)
        predictions_2 = model_2.predict(X_test)

        mse_OLS = mean_squared_error(y_test, predictions_2)
        r2_OLS = r2_score(y_test, predictions_2)
        print("MSE Regression = %f, MSE MLP = %f" % (mse_OLS, mse_MLP))
        print("R2 Regression = %f, R2 MLP = %f" % (r2_OLS, r2_MLP))
```

```
In [ ]: train_size = int(len(X) * (1 - test_size))
        X_train, X_test = X[0:train_size], X[train_size : len(X)]
        y_train, y_test = Y[0:train_size], Y[train_size : len(X)]

        modelMLP = MLPRegressor(hidden_layer_sizes=(50,))
        modelOLS = LinearRegression()
        model_MLP = modelMLP.fit(X_train, y_train)
        model_OLS = modelOLS.fit(X_train, y_train)

        Y_predMLP = pd.DataFrame(
            model_MLP.predict(X_test), index=y_test.index, columns=y_test.columns
        )

        Y_predOLS = pd.DataFrame(
```

```
model_OLS.predict(X_test), index=y_test.index, columns=y_test.columns
)
```

Let's compare the two predictions below:

```
In [ ]: pd.DataFrame(
        {
            "Actual : 1m": y_test.loc[:, "1m_pred"],
            "Prediction MLP 1m": Y_predMLP.loc[:, "1m_pred"],
            "Prediction OLS 1m": Y_predOLS.loc[:, "1m_pred"],
        }
    ).plot(figsize=(10, 5))
    pyplot.title("1 month rate prediction")
    pyplot.ylabel("Yield Rates")
    pyplot.suptitle(
        "Fig. 16: 1 Month Rate Prediction",
        fontweight="bold",
        horizontalalignment="right",
        fontsize=15,
    )
    pyplot.show
```

```
In [ ]: pd.DataFrame(
        {
            "Actual : 5yr": y_test.loc[:, "5y_pred"],
            "Prediction MLP 5yr": Y_predMLP.loc[:, "5y_pred"],
            "Prediction OLS 5yr": Y_predOLS.loc[:, "5y_pred"],
        }
    ).plot(figsize=(10, 5))
    pyplot.title("5 years rate prediction")
    pyplot.ylabel("Yield Rates")
    pyplot.suptitle(
        "Fig. 17: 5 Years Prediction",
        fontweight="bold",
        horizontalalignment="right",
        fontsize=15,
    )
    pyplot.show
```

```
In [ ]: pd.DataFrame(
        {
            "Actual : 30yr": y_test.loc[:, "30y_pred"],
            "Prediction MLP 30yr": Y_predMLP.loc[:, "30y_pred"],
            "Prediction OLS 30yr": Y_predOLS.loc[:, "30y_pred"],
        }
    ).plot(figsize=(10, 5))
    pyplot.title("30 years rate prediction")
    pyplot.ylabel("Yield Rates")
    pyplot.suptitle(
        "Fig. 18: 30 Year Prediction",
        fontweight="bold",
        horizontalalignment="right",
        fontsize=15,
    )
    pyplot.show
```

From the charts above, the linear regression model seems to provide a better prediction of yield rates as compared to the ANN model.

## 5. Conclusion

In this lesson, we have applied an ordinary linear regression and a deep learning model to forecast yield curve in all short, medium, and long terms. Although the linear regression model outperformed the MLP model in this exercise, there are a variety of problems in which a deep learning model will give us superior results, and it is therefore advisable to try out a couple of models before settling on one. We have also seen that we can always improve the performance of neural networks by performing a grid search to select optimal parameters.

The output of the deep learning model can be improved by finding the right hyperparameters and increasing the layers of the model. In the next lesson, we will look at optimization algorithms in deep learning, the challenges they face, how they affect output of our models, and how to address these challenges.

### References

1. Bernadell, Carlos, et al. "Yield Curve Prediction for the Strategic Investor." European Central Bank, *Working Paper Series*, no. 472, 2005.
2. Bluwstein, Kristina, et al. "Credit Growth, the Yield Curve and Financial Crisis Prediction: Evidence from a Machine Learning Approach." (2021).
3. Hyndman, Cody. "Arbitrage-Free Yield Curve and Bond Price Forecasting by Deep Neural Networks." *YouTube*, uploaded by Fields Institute, 29 Sept. 2021.
4. Arnold, Ludovic, et al. "An Introduction to Deep Learning." European Symposium on Artificial Neural Networks (ESANN), 2011.